Analisi Tecnica del Firmware

Progetto *Vera-Module-RBL-XE*

Analisi del progetto a cura di RetroBit Lab

21 ottobre 2026

Indice

1 Introduzione **1**

2 Architettura del Sistema **1**
2.1 Componenti Chiave 1
2.2 Modalità operative (VERA_BOARD_IS_PBI) 1
2.3 Ambiente di build PlatformIO 2
2.4 Interfacciamento Bus 2

3 Mapping dei Pin (ESP32-S3FN8, QFN56) **3**
3.1 Pin di strapping — comportamento al boot con bus ATARI connesso . . . 5
3.2 Quarzo oscillatore 40 MHz (pin 53–54) 6

4 Analisi delle Tempistiche **7**
4.1 Accesso diretto ai registri GPIO 8
4.2 Diagramma Temporale di un Ciclo di Lettura (PBI) 8

5 Frammento di Codice Sorgente **8**

6 Diagramma di Flusso del Firmware **11**

7 Verifiche hardware emerse dal lavoro su driver ed emulatore **12**

8 Conclusione **14**

1 Introduzione

Questo documento presenta un'analisi tecnica del firmware per il progetto *Vera-Module-RBL-XE*. L'obiettivo del firmware, eseguito su un microcontrollore ESP32, è di interfacciarsi con il bus di un processore 6502 per emulare un dispositivo PBI (Parallel Bus Interface) o CCTL (Cartridge Control Line) per computer ATARI della serie XL/XE.

Il progetto supporta la MCU target ESP32-S3 e due modalità operative, per un totale di due varianti di compilazione distinte. L'analisi si concentra sulla fattibilità temporale (timing) dell'interazione tra la MCU moderna e la CPU vintage, illustrando la logica di funzionamento tramite diagrammi di flusso e temporali.

2 Architettura del Sistema

Il sistema si basa sull'interazione di due componenti principali.

2.1 Componenti Chiave

- **CPU 6502 (MOS/SALLY):** Il processore centrale dell'Atari, operante a una frequenza di clock di **1.7897 MHz** (NTSC).
- **MCU:** Il microcontrollore moderno responsabile del monitoraggio del bus e dell'emulazione della periferica. Il target è:
  - **ESP32-S3FN8** (QFN56, 7×7 mm) — Dual-core Xtensa LX7 @ 240 MHz, 512 KB SRAM, 8 MB flash in-package Quad SPI, 45 GPIO bidirezionali.

2.2 Modalità operative (VERA_BOARD_IS_PBI)

Il firmware supporta due modalità operative, selezionate a compile-time tramite la macro `VERA_BOARD_IS_PBI` in `main.cpp`:

- **PBI mode (`VERA_BOARD_IS_PBI=1`, default):** Il modulo si collega al connettore ECI della serie XL/XE.
  - **VCS latch ($D1FF):** in scrittura il dispositivo usa solo il *proprio* bit dati (D7): D7 = 1 seleziona, D7 = 0 deseleziona (`data & 0x80`). In lettura $D1FF non è pilotato: l'identificazione dell'IRQ via $D1FF non è realizzabile con `U19` (vedere sezione 7).
  - **ROM $D800–$DFFF (2 KB, 11 bit A0–A10):** `MPD` (Math Pack Disable, GPIO42) viene asserito basso per ogni ciclo ROM con device selezionato, disabilitando la ROM Math Pack interna all'Atari.
  - **RAM $D600–$D7FF (512 B, 9 bit A0–A8):** buffer IRAM esposto al 6502. `EXTSEL_N` viene asserito per-ciclo.
  - **Registri VERA $D100–$D11F** (32 registri): `DEV_SEL_N` e `EXTSEL_N` affermati *sempre*, indipendentemente dal latch; la FPGA VERA guida il bus dati. L'OS deseleziona la scheda ($D1FF = 0) subito dopo l'`INIT` e il driver non

la riseleziona mai (selezionarla mapperebbe la ROM e disattiverebbe il Math Pack, rompendo la virgola mobile del BASIC): i registri devono quindi restare raggiungibili con il latch a 0.

  - **Intervallo $D120–$D1FE**: `EXTSEL_N` asserito solo con scheda selezionata; nessun dato guidato dall'ESP32 (non implementato nel firmware corrente).
  - **EXTSEL_N (GPIO41):** segnale *per-ciclo* che disabilita MMU/Freddie dell'Atari per accessi D1xx e D6xx. Non è un segnale persistente.

- **CCTL mode (`VERA_BOARD_IS_PBI=0`):** Il modulo si collega alla porta cartuccia. Il firmware decodifica l'indirizzo completo a 16 bit e intercetta $D500–$D5FF: `DEV_SEL_N` affermato per $D500–$D5FE (FPGA guida il bus dati), latch di selezione a $D5FF (scrittura di $80 seleziona la scheda).

2.3 Ambiente di build PlatformIO

Il file `platformio.ini` definisce tre ambienti:

Ambiente	Modalità	Note
`esp32s3fn8`	PBI ($D1FF)	produzione, scheda senza modifiche
`esp32s3fn8-cctl`	CCTL ($D5FF)	slot cartuccia

Opzioni di compilazione (`-D`):

Macro	Default	Effetto
`VERA_BOARD_IS_PBI`	1	1 = PBI, 0 = CCTL (impostata dall'ambiente)
`VERA_TRACE_REGS`	0	log di ogni accesso ai registri VERA. **Solo debug**: costa ≈1 µs nel loop del bus (un ciclo 6502 dura 559 ns) e può lasciare `DEV_SEL_N` asserito nel ciclo successivo con accessi spuri alla VERA

2.4 Interfacciamento Bus

L'ESP32 legge il bus indirizzi completo **A0–A15** e i segnali `PHI2` e `R/W_` direttamente dai propri GPIO ad ogni ciclo di clock. **Non esiste alcuna logica esterna di pre-decodifica degli indirizzi**: i segnali `D1XX_N`, `ROM_SEL_N` e `RAM_SEL_N` del progetto precedente sono stati eliminati. La decodifica è interamente software in `decode_addr()`, che restituisce un indirizzo a 16 bit senza ambiguità di pagina.

L'adattamento di livello 5 V ↔ 3.3 V è affidato a cinque **74LVC4245** (lato A = Atari 5 V, lato B = scheda 3.3 V; direzione cablata, tranne `U19`) e a due porte **SN74HCT04** (`U23`). I TXS0108E delle revisioni precedenti sono stati eliminati (il 74LVC4245 non ha auto-sensing). Riferimenti secondo la netlist del 2026-09-01; dettaglio in `PIN-MAPPING.md`:

- **U19 — Bus Dati D0–D7 (GPIO 4–11):** bidirezionale. `DIR` = NOT R/W_ e `~OE_` = NOT PHI2, entrambi tramite `U23`. Il firmware abilita le uscite ESP32 con `GPIO.enable_w1ts` solo nei cicli che richiedono risposta dell'ESP32 (ROM $D800–$DFFF, RAM $D600–$D7FF, RAMbo $4000–$7FFF). **`~OE_` oggi è NOT(PHI2) e va sostituito con il gating di sezione 7.**
- **U4 — Bus Indirizzi A0–A7 (GPIO 12–19):** ingressi Atari→ESP32. GPIO 19 condivide il pad USB D-; USB CDC è disabilitato nel firmware.
- **U3 — Bus Indirizzi A8–A15 (GPIO 20–21, 35–36, 33–34, 45–46):** ingressi. Applicare *length matching* (serpentine alla traccia più lunga) ad A12–A15; GPIO 33–34 e GPIO 45–46 sono su lati opposti del QFN56.
- **U6 — PHI2, R/W_ (GPIO 1, 2), più D1XX, S4, S5, CCTL, REFRESH:** ingressi Atari→scheda. I cinque segnali pre-decodificati sono portati a test point e *non* collegati a GPIO.
- **U16 — EXTSEL_N, MPD, ARESET (GPIO 41, 42, 37), `~mVIRQ` della FPGA, più HALT, RD4, RD5:** uscite scheda→Atari. HALT, RD4 e RD5 sono a test point. **L'IRQ della VERA è attualmente un'uscita push-pull sul bus** (vedere sezione 7). I GPIO 39–42 erano riservati a JTAG: `setup()` li de-registra con `gpio_reset_pin()` prima di creare il Dedicated GPIO bundle.
- **U5 (74HC00) — strobe VERA:** `~mRE` = NAND(PHI2, R/W_), `~mWE` = NAND(PHI2, NOT R/W_) verso la FPGA VERA, che richiede RD_n e WR_n distinti.
- **Connessioni dirette (senza level shifter, 3.3 V):** `DEV_SEL_N` (GPIO 40) → `~mVCS0` (VERA /CS); `CRESET` (GPIO 38) → reset VERA; `CDONE` (GPIO 39) ← VERA CDONE.

I segnali `DEV_SEL_N`, `EXTSEL_N` e `MPD` sono raggruppati nel **Dedicated GPIO output bundle** (Core 0, canali 0–2): vengono asseriti e de-asseriti in un'unica istruzione TIE Xtensa (`ee.wr_mask_gpio_out`, ≈1 ciclo CPU), anziché con tre scritture APB separate (≈18 cicli totali). Il segnale `PHI2` è nel bundle di ingresso (canale 0): il polling avviene via `ee.get_gpio_in` (1 ciclo CPU).

3 Mapping dei Pin (ESP32-S3FN8, QFN56)

I pin fisici del package QFN56 sono numerati 1–56 (più il pad GND centrale). La tabella seguente riporta i segnali del progetto con il numero GPIO logico **e** il numero di pad fisico del QFN56 (da Table 2-1 del datasheet ESP32-S3 v2.2).

Nota importante: il MCU *richiede* un **quarzo esterno a 40 MHz** collegato ai pin dedicati analogici **XTAL_P** (pin 54) e **XTAL_N** (pin 53). L'oscillatore RC interno è limitato al dominio RTC e non è in grado di pilotare il PLL della CPU a 240 MHz; senza quarzo il chip non si avvia. Vedere la sezione 3.2 per il circuito consigliato.

Segnale	GPIO	Pin QFN56	IO MUX	Dir.	Note
Bus dati D0–D7 — bidirezionale via 74LVC4245 U19 (Bank-0 bit 4–11)					
D0	4	9	GPIO4	I/O	glitch LOW 60 µs al power-up
D1	5	10	GPIO5	I/O	
D2	6	11	GPIO6	I/O	
D3	7	12	GPIO7	I/O	
D4	8	13	GPIO8	I/O	
D5	9	14	GPIO9	I/O	
D6	10	15	GPIO10	I/O	
D7	11	16	GPIO11	I/O	
Bus indirizzi A0–A7 — input via 74LVC4245 U4 (Bank-0 bit 12–19)					
A0	12	17	GPIO12	IN	
A1	13	18	GPIO13	IN	
A2	14	19	GPIO14	IN	
A3	15	21	XTAL_32K_P	IN	pad RTC usato come GPIO; pin 20 = VDD3P3_RTC
A4	16	22	XTAL_32K_N	IN	pad RTC usato come GPIO
A5	17	23	GPIO17	IN	
A6	18	24	GPIO18	IN	
A7	19	25	USB_D−	IN	USB CDC disabilitato
Bus indirizzi A8–A11 (U3) e controllo (U6: PHI2, R/W_; U16: EXTSEL_N, MPD, ARESET)					
A8	20	26	USB_D+	IN	USB CDC disabilitato
A9	21	27	GPIO21	IN	
A10	35	40	GPIO35	IN	Bank-1 bit 3
A11	36	41	GPIO36	IN	Bank-1 bit 4
PHI2	1	6	GPIO1	IN	Dedicated GPIO input ch 0; 1.79 MHz
R/W_	2	7	GPIO2	IN	HIGH = lettura
EXTSEL_N	41	47	MTDI	OUT	Dedicated GPIO output ch 1; ex-JTAG, reclaimed
MPD	42	48	MTMS	OUT	Dedicated GPIO output ch 2; ex-JTAG, reclaimed
ARESET	37	42	GPIO37	OUT	Reset sistema Atari (LOW asserito)
Bus indirizzi A12–A15 — input via 74LVC4245 U3 (Bank-1)					
A12	33	38	GPIO33	IN	Bank-1 bit 1; length matching con A13–A15
A13	34	39	GPIO34	IN	Bank-1 bit 2
A14	45	51	GPIO45	IN	Bank-1 bit 13; pin di strapping, usato come A14
A15	46	52	GPIO46	IN	Bank-1 bit 14; pin di strapping, usato come A15
Segnali diretti (senza level shifter, 3.3 V)					
DEV_SEL_N	40	45	MTDO	OUT	VERA /CS; Dedicated GPIO output ch 0; ex-JTAG
CRESET	38	43	GPIO38	OUT	VERA /RESET
CDONE	39	44	MTCK	IN	VERA configurata (HIGH = OK); ex-JTAG
RAMBO_EN	3	8	GPIO3	IN	RAMbo enable: pull-up 10 kΩ→VCC = presente
BOOT0	0	5	GPIO0	IN	pulsante BOOT / auto-reset (rete `BOOT0`); non usato dal firmware
Debug					
UART TX	43	49	U0TXD	OUT	Serial debug @ 115200 baud
UART RX	44	50	U0RXD	IN	non utilizzato in firmware

Decodifica indirizzo (16 bit)

`decode_addr()` restituisce l'indirizzo completo A0–A15 (16 bit), catturato in due letture APB (`GPIO.in` per A0–A9 e `GPIO.in1.val` per A10–A15) già eseguite per il campionamento di PHI2. Le maschere di accesso ai buffer IRAM sono:

ROM \$D800–\$DFFF : `pbi_driver[addr & 0x7FF]` (2048 B, 11 bit A0–A10)

RAM \$D600–\$D7FF : `ram_pbi[addr & 0x1FF]` (512 B, 9 bit A0–A8)

RAMbo \$4000–\$7FFF :

`extended_rambo_256k[(bank` $\ll 14$) | (addr & 0x3FFF)] (256 KB, 16 banchi)

GPIO disponibili

Con A12–A15 aggiunti e `RAMBO_EN` (GPIO 3), tutti i GPIO utilizzabili sono ora assegnati. Non rimangono GPIO liberi per usi futuri.
Attenzione: I pin QFN56 28 e 30–35 (GPIO 26–32) sono collegati internamente alla flash Quad SPI in-package (variante FN8) e non devono mai essere connessi esternamente. GPIO 45 (pin 51) e GPIO 46 (pin 52) sono pin di strapping del chip: vengono usati come ingressi A14 e A15 (vedere sezione 3.1). I pin 53 (`XTAL_N`) e 54 (`XTAL_P`) sono pad analogici dedicati al quarzo 40 MHz: riservati, non usabili come GPIO (vedere sezione 3.2).

3.1 Pin di strapping — comportamento al boot con bus ATARI connesso

L'ESP32-S3 campiona quattro pin di strapping durante la finestra di reset al power-on, *prima che qualsiasi codice venga eseguito*. I livelli campionati configurano la modalità di boot e alcune opzioni di sistema.

Considerazione temporale critica. `PIN_ARESET` (GPIO 37) viene configurato come `OUTPUT` soltanto all'interno di `setup()`. Al momento del campionamento dei pin di strapping GPIO 37 è ancora in alta impedenza: l'ESP32 **non ha ancora asserito** la linea di reset dell'Atari. È la circuiteria di power-on reset dell'Atari stessa (rete RC di MMU/-POKEY) che mantiene il pin `RESET#` del 6502 basso, mettendo il bus indirizzi in alta impedenza. I transceiver 74LVC4245, con gli ingressi lato Atari flottanti, *non* garantiscono uscite in alta impedenza (non hanno auto-sensing né bus-hold): per A14 e A15 il livello campionato dipende dal pull-down interno dell'ESP32 contro un nodo non pilotato. **Da verificare sulla scheda**; se necessario aggiungere un pull-down da 10 kΩ lato ESP32 su GPIO 45 e GPIO 46.

- **GPIO 0 — Pin 5 | Boot mode | Pull-UP interno ≈45 kΩ**
  Rete `BOOT0`: `R33` 4.7 kΩ verso 3V3, pulsante `SW3`, `C61` 100 nF e `R54` 220 Ω verso `Q4` (auto-reset del programmatore). Non connesso al bus ATARI. Il pull-up mantiene GPIO 0 HIGH per tutta la finestra di campionamento. Valore campionato: **HIGH → SPI boot dalla flash integrata** (comportamento corretto per il normale funzionamento). Il valore LOW avvierebbe la modalità download UART/JTAG, bloccando il sistema.

- **GPIO 3 — Pin 8 | JTAG source | Nessun pull interno (floating)**
  Connesso a `RAMBO_EN` tramite resistore esterno 10 kΩ verso VCC o GND. Il resistore esterno determina il livello prima del campionamento:

  - Pull-up (RAMbo presente) → **HIGH** → sorgente JTAG hardware selezionata (GPIO 39–42). *Ininfluente:* `gpio_reset_pin()` reclaima tutti i pin JTAG in `setup()` prima della creazione del bundle Dedicated GPIO.
  - Pull-down (RAMbo assente) → **LOW** → JTAG via USB OTG. *Ininfluente:* `ARDUINO_USB_CDC_ON_BOOT=0` disabilita USB CDC; GPIO 19/20 sono riassegnati ad A7/A8.

Senza il resistore esterno il pin flotta e il valore campionato è indeterminato: il resistore da 10 kΩ è **obbligatorio** per un boot deterministico.

- **GPIO 45 — Pin 51 | VDD_SPI voltage | Pull-DOWN interno ≈5 kΩ**
  Connesso al bit A14 del bus indirizzi ATARI tramite 74LVC4245 U3 (5 V ↔ 3.3 V). Al power-on il `RESET#` del 6502 è basso → il pin A14 lato Atari è in alta impedenza; l'uscita lato ESP32 di U3 non è pilotata in modo garantito (vedere sopra) e il pull-down interno porta GPIO 45 a LOW.
  Valore campionato: **LOW → VDD_SPI alimentato dal LDO interno (≈1.8 V)**. Per la variante FN8 (flash integrata on-package) il rail VDD_SPI è instradato internamente e indipendente dallo strapping; entrambi i valori LOW e HIGH sono elettricamente sicuri. Il pull-down è coerente con lo stato idle del bus. ✓

  Rischio residuo: se un accoppiamento parassita o una salita particolarmente rapida del rail 5 V Atari porta A14 HIGH durante la finestra di campionamento, il risultato è VDD_SPI = 3.3 V (VDD3P3_RTC) — anch'esso sicuro per FN8.

- **GPIO 46 — Pin 52 | ROM messages (UART0) | Pull-DOWN interno ≈5 kΩ**
  Connesso al bit A15 del bus indirizzi ATARI tramite 74LVC4245 U3. Scenario di power-on identico a GPIO 45: 6502 in reset → A15 alta impedenza → ingresso U3 non pilotato → pull-down → GPIO 46 LOW.
  Valore campionato: **LOW → messaggi ROM del primo stage su UART0 disabilitati**. Corretto per la produzione: il firmware inizializza UART0 autonomamente tramite `Serial.begin()` e lo usa esclusivamente per l'output di debug. L'abilitazione dei messaggi ROM (HIGH) aggiungerebbe righe di log Espressif prima dell'output del firmware — indesiderato ma non dannoso. ✓

3.2 Quarzo oscillatore 40 MHz (pin 53–54)

L'ESP32-S3FN8 richiede un quarzo esterno da **40 MHz** per generare il riferimento PLL interno che alimenta la CPU a 240 MHz, l'interfaccia flash e i divisori UART. I pin sono *analogici dedicati*: non configurabili via software, non condivisi con GPIO.

Pin QFN56	Segnale	Note
53	`XTAL_N`	Terminale negativo del cristallo (uscita amplificatore oscillatore)
54	`XTAL_P`	Terminale positivo del cristallo (ingresso amplificatore oscillatore)

Schema consigliato:

```
XTAL_P (54) --[C1 10pF]-- GND
      |
   [X1 40MHz]    <- Rs 0-33 Ohm in serie (opzionale)
      |
XTAL_N (53) --[C2 10pF]-- GND
```

Componente	Valore	Note
X1 – quarzo	40 MHz, SMD 3225 o 2016	±10 ppm; verificare C_L dal datasheet
C1, C2 – condensatori di carico	10 pF, 0402, ±5 %	$C_{ext} = 2C_L - C_{stray}$; $C_{stray} \approx 3$–5 pF
Rs – resistenza serie	0–33 Ω (opzionale)	Limita il drive dell'oscillatore

Regole di layout PCB: tracce XTAL_P/N più corte possibile (<5 mm); condensatori di carico posizionati ai pad del chip, non al corpo del quarzo; piano di massa solido sotto l'area del quarzo; isolamento dalle tracce PHI2, clock CPU 240 MHz e bus dati D0–D7.

4 Analisi delle Tempistiche

Una delle principali preoccupazioni in un progetto di questo tipo è la capacità della MCU di rispondere entro la finestra temporale definita dal ciclo di clock della CPU 6502.

- **Ciclo Clock 6502 (1.7897 MHz, NTSC):** $T_{6502} = 1/(1.7897 \times 10^6) \approx 559\,\text{ns}$
- **Finestra PHI2 alto (half-cycle):** $T_{PHI2\uparrow} \approx 279\,\text{ns}$ — questa è la finestra critica entro cui l'ESP32 deve leggere l'indirizzo, decodificarlo ed eventualmente pilotare il bus dati.
- **Ciclo Clock ESP32 (240 MHz):** $T_{ESP32} = 1/(240 \times 10^6) \approx 4.17\,\text{ns}$

Il rapporto tra la finestra critica e il ciclo dell'ESP32 è:

$$\frac{T_{PHI2\uparrow}}{T_{ESP32}} = \frac{279\,\text{ns}}{4.17\,\text{ns}} \approx 67\,\text{cicli ESP32}$$

Anche considerando solo la semi-finestra di PHI2 alto (il caso peggiore), l'ESP32-S3 dispone di circa **67 cicli di clock** per completare tutta la catena: lettura di `GPIO.in` / `GPIO.in1.val`, decodifica dell'indirizzo, asserzione dei segnali di controllo, pilotaggio del bus dati.

La versione precedente decodificava l'indirizzo con confronti di range a ogni ciclo: dal disassemblato risultavano circa **95 istruzioni** fra le letture APB e l'asserzione dei segnali di controllo, oltre la finestra disponibile. La versione corrente usa tabelle precalcolate (`include/bus_decode.h`): la pagina A8–A15, presa nell'ordine dei pin, indicizza una tabella di 256 voci per lo stato corrente (latch selezionato × RAMbo attivo) con i livelli di `DEV_SEL_N`/`EXTSEL_N`/`MPD` e le azioni da eseguire. Il percorso critico è ora di **16 istruzioni** dopo le letture APB (26 per `$D1xx`/`$D5xx`, che dipendono anche da A0–A7) e circa 22 ulteriori per il dato in lettura. Il codice e i dati del hot loop sono rispettivamente in IRAM e DRAM; i tempi reali (letture APB, sincronizzatori GPIO) vanno confermati con un analizzatore logico.

Nei cicli di scrittura il dato del 6502 è valido solo ≈100–150 ns dopo la salita di PHI2: il firmware rilegge `GPIO.in` finché PHI2 è alto (PHI2 è nello stesso registro) e usa l'ultimo campione, poi applica gli effetti (latch, PORTB/PBCTL, RAM PBI, RAMbo) durante PHI1.

4.1 Accesso diretto ai registri GPIO

Per eliminare qualsiasi overhead di astrazione, il firmware usa esclusivamente accesso diretto ai registri hardware dell'ESP32:

- `GPIO.in` — legge i 32 GPIO del bank-0 in un'unica istruzione a 32 bit.
- `GPIO.in1.val` — legge i GPIO del bank-1 (GPIO32–48).
- `GPIO.out_w1ts` / `GPIO.out_w1tc` — set/clear atomico dei bit di uscita (write-1-to-set / write-1-to-clear).
- `GPIO.enable_w1ts` / `GPIO.enable_w1tc` — abilitazione/disabilitazione in uscita (tristate) per il bus dati.

4.2 Diagramma Temporale di un Ciclo di Lettura (PBI)

Il diagramma seguente illustra un tipico ciclo di lettura gestito dall'ESP32 in modalità PBI.

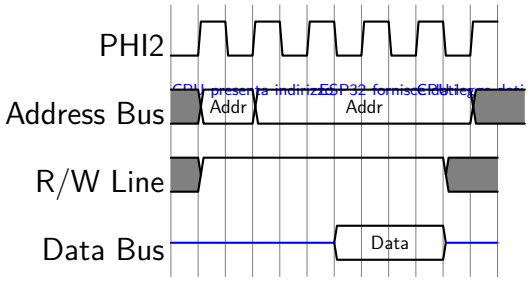


Figura 1: Diagramma temporale di un ciclo di lettura del 6502 gestito dall'ESP32 (modalità PBI).

5 Frammento di Codice Sorgente

Di seguito è riportato un estratto aggiornato del file `main.cpp` che mostra le definizioni dei pin e il cuore del `MonitorTask`. Tutte le funzioni critiche sono marcate `IRAM_ATTR` per garantire l'esecuzione dalla RAM interna. L'indirizzo completo A0–A15 è decodificato interamente in software: nessuna logica esterna 74HCT/74LVC.

```
/* Bank 0 (GPIO.in) */
#define PIN_PHI2        1    /* GPIO1, pin 6 -- Dedicated GPIO input ch 0  */
#define PIN_RW          2    /* GPIO2, pin 7 -- R/W_, HIGH = read */
/* D0-D7: GPIO 4-11, bank-0 bit 4-11 */
#define DBUS_MASK       0x00000FF0UL
/* A0-A9: GPIO 12-21, bank-0 bit 12-21 */
#define ABUS_LO_MASK    0x003FF000UL

/* Bank 1 (GPIO.in1.val) */
```

```
#define PIN_A10          35   /* GPIO35, pin 40  -- bank-1 bit 3  */
#define PIN_A11          36   /* GPIO36, pin 41  -- bank-1 bit 4  */
#define PIN_A12          33   /* GPIO33, pin 38  -- bank-1 bit 1  */
#define PIN_A13          34   /* GPIO34, pin 39  -- bank-1 bit 2  */
#define PIN_A14          45   /* GPIO45, pin 51  -- bank-1 bit 13 */
#define PIN_A15          46   /* GPIO46, pin 52  -- bank-1 bit 14 */
#define PIN_ARESET       37   /* GPIO37, pin 42  -- system reset  */
#define PIN_CRESET       38   /* GPIO38, pin 43  -- VERA reset    */
#define PIN_CDONE        39   /* GPIO39, pin 44  -- VERA done     */
#define PIN_DEV_SEL_N    40   /* GPIO40, pin 45  -- Dedicated out ch 0 */
#define PIN_EXTSEL_N     41   /* GPIO41, pin 47  -- Dedicated out ch 1 */
#define PIN_MPD          42   /* GPIO42, pin 48  -- Dedicated out ch 2 */
#define PIN_RAMBO_EN      3   /* GPIO3,  pin  8  -- RAMbo enable  */

/* Dedicated GPIO channel masks */
#define DEDIC_IN_PHI2        (1u << 0)
#define DEDIC_OUT_DEVSELN    (1u << 0)
#define DEDIC_OUT_EXTSEL     (1u << 1)
#define DEDIC_OUT_MPD        (1u << 2)
#define DEDIC_OUT_CTRL_MASK 0x07u
#define DEDIC_OUT_CTRL_IDLE 0x07u  /* tutti HIGH = deasserted */

/* include/bus_decode.h: indice di pagina A8-A15 nell'ordine dei pin
 *   bit 0,1 = A8,A9 (GPIO20,21)   bit 2,3 = A12,A13 (GPIO33,34)
 *   bit 4,5 = A10,A11 (GPIO35,36) bit 6,7 = A14,A15 (GPIO45,46)   */
static inline uint32_t bus_page_idx(uint32_t lo, uint32_t hi)
{
    return ((lo >> 20) & 0x03u) | ((hi << 1) & 0x3Cu) | ((hi >> 7) & 0
   xC0u);
}

/* Voce di tabella: bit 0-2 livelli DEV_SEL_N/EXTSEL_N/MPD, poi i flag
   */
#define CT_SLOW    (1u << 3)   /* $D1xx/$D5xx: dipende anche da A0-A7 */
#define CT_DRIVE   (1u << 4)   /* lettura: pilota rd_base[idx][A0-A7] */
#define CT_RAMBO   (1u << 5)   /* + offset del banco RAMbo corrente   */
#define CT_WR_MEM  (1u << 6)   /* scrittura: memorizza in rd_base     */
#define CT_WR_REG  (1u << 7)   /* scrittura: latch / snoop PIA        */
```

Listing 1: Definizioni pin ESP32-S3FN8 (QFN56) – firmware corrente

```
static void IRAM_ATTR MonitorTask(void *arg)   /* Core 0 */
{
    /* Bundle Dedicated GPIO: input [PHI2], output [DEV_SEL_N, EXTSEL_N,
    MPD] */
    /* ... (setup bundle e sequenza di reset omessi per brevita') ... */
    bool           selected  = false;
    uint8_t        portb = 0xFF, pbctl = 0x00;
    bool           rambo_on  = false;
    const uint8_t *ctrl_cur  = bus_ctrl[bus_state(selected, rambo_on)];
    uint32_t       rambo_off = bus_rambo_bank(portb) << 14;

    for (;;) {
        /* 1. Attesa fronte di salita PHI2 via TIE (1 ciclo CPU) */
        while (!(cpu_ll_read_dedic_gpio_in() & DEDIC_IN_PHI2));
        uint32_t g_lo = GPIO.in;
```

```
15          uint32_t g_hi = GPIO.in1.val;
16
17          /* 2. Lookup di pagina: livelli di controllo e azioni */
18          uint32_t idx  = bus_page_idx(g_lo, g_hi);
19          uint32_t c    = ctrl_cur[idx];
20          uint32_t off8 = (g_lo >> 12) & 0xFFu;
21          if (c & CT_SLOW)
22              c = bus_slow_ctrl(c, off8, selected, is_pbi);
23
24          /* 3. Asserzione segnali controllo (1 istruzione TIE) */
25          cpu_ll_write_dedic_gpio_mask(DEDIC_OUT_CTRL_MASK, c);
26
27          if (g_lo & (1UL << PIN_RW)) {
28              /* 4a. Lettura: ROM PBI / RAM PBI / banco RAMbo */
29              if (c & CT_DRIVE) {
30                  const uint8_t *p = bus_rd_base[idx];
31                  if (c & CT_RAMBO) p += rambo_off;
32                  bus_drive(p[off8]);
33              }
34              while (cpu_ll_read_dedic_gpio_in() & DEDIC_IN_PHI2);
35              if (c & CT_DRIVE) bus_release();
36              cpu_ll_write_dedic_gpio_mask(DEDIC_OUT_CTRL_MASK,
    DEDIC_OUT_CTRL_IDLE);
37              continue;
38          }
39
40          /* 4b. Scrittura: ultimo campione con PHI2 ancora alto */
41          uint32_t last = g_lo, now;
42          while ((now = GPIO.in) & (1UL << PIN_PHI2)) last = now;
43          cpu_ll_write_dedic_gpio_mask(DEDIC_OUT_CTRL_MASK,
    DEDIC_OUT_CTRL_IDLE);
44          uint8_t data = decode_data(last);
45
46          /* 5. Effetti durante PHI1 */
47          if (c & CT_WR_MEM) {                        /* RAM PBI o RAMbo */
48              uint8_t *p = (uint8_t *)bus_rd_base[idx];
49              if (c & CT_RAMBO) p += rambo_off;
50              p[off8] = data;
51          }
52          if (c & CT_WR_REG) {
53              uint8_t page = bus_idx_to_page(idx);
54              if (off8 == 0xFF && page == (is_pbi ? 0xD1 : 0xD5)) {
55                  selected = (data & PBI_DEV_ID) != 0;   /* + log_send()
    */
56              } else if (page == 0xD3) {          /* snoop PIA (passivo) */
57                  if (off8 == 0x03) pbctl = data;
58                  else if (off8 == 0x01 && (pbctl & 0x04)) {
59                      portb     = data;
60                      rambo_off = bus_rambo_bank(portb) << 14;
61                      rambo_on  = rambo_hw_enabled && !(portb & 0x10);
62                  }
63              }
64              ctrl_cur = bus_ctrl[bus_state(selected, rambo_on)];
65          }
66      }
```

```
67 }
```

Listing 2: Nucleo del MonitorTask – hot loop semplificato (PBI + RAMbo)

6 Diagramma di Flusso del Firmware

La logica principale del firmware è contenuta nel `MonitorTask`, che viene eseguito da solo sul Core 0 con priorità massima e non cede mai la CPU (l'idle task del Core 0 è tolto dal task watchdog). Il Core 1 esegue `setup()`/`loop()`: svuota il ring buffer degli eventi (`log_ring`, single-producer/single-consumer senza lock) e li stampa su `Serial`.

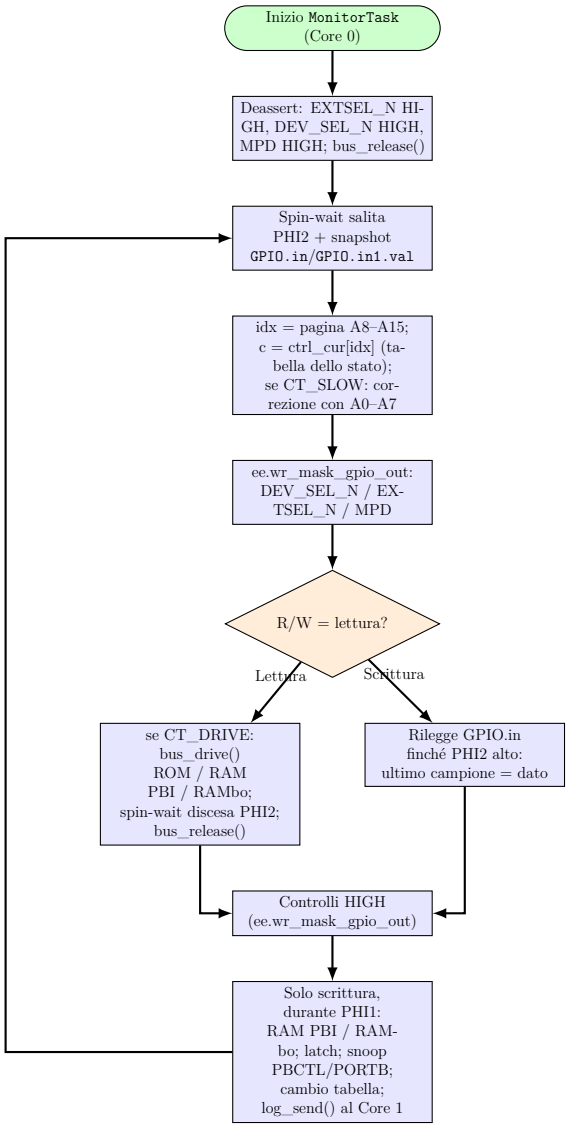


Figura 2: Diagramma di flusso semplificato del `MonitorTask` (modalità PBI, ESP32-S3FN8).

7 Verifiche hardware emerse dal lavoro su driver ed emulatore

Il lavoro sul driver 6502 e sull'emulatore (confrontato con l'HDL della VERA 48.0.1, che per il bus coincide con la 47.0.2 salvo la cattura dell'indirizzo di scrittura direttamente all'inizio di WR) ha evidenziato requisiti e rischi per la scheda; il dettaglio, con il piano di bring-up, è in `VERA-ATARI-HW-REQUIREMENTS.md`. I punti seguenti derivano dalla *netlist* (nessuna simulazione elettrica né misura) e vanno confermati sullo schema aggiornato e con l'oscilloscopio.

Cosa richiede la VERA sul bus

- RD_n e WR_n **distinti**: `bus_read = !cs_n && wr_n && !rd_n`, `bus_write = !cs_n && !wr_n`.
- In scrittura l'indirizzo è campionato sul fronte di *salita* di `bus_write` e il **dato sul fronte di discesa** (fine scrittura): il dato deve restare stabile *dopo* la salita di WR_n/CS_n (tempo di hold).
- In lettura gli effetti collaterali (autoincremento, prefetch) avvengono sulla fine di RD_n (circa 120 ns di sincronizzazione): ogni attivazione di RD_n è un evento, quindi niente cicli doppi (RMW, dummy read) sulle porte DATA0/DATA1.
- **Nessun pin di reset**: solo un POR interno (128 cicli di `clk25`). La scrittura di CTRL bit 7 ($D105) **riconfigura l'intero FPGA** (VRAM persa, bus morto ≈100 ms).
- `IRQ_n` è **push-pull** a livello. I registri FX sono in sola scrittura (la lettura restituisce i byte di identità del firmware, `'V',48,0,1` sulla 48.0.1).

Punti aperti sulla scheda attuale

- **Priorità A — possibile contesa sul bus dati.** `U19` ha `~OE_` = NOT(PHI2) *senza* dipendere dalla selezione: in ogni lettura (anche di RAM/ROM Atari) il transceiver pilota il bus Atari dal lato scheda, dove GPIO ESP32 e uscite FPGA sono Hi-Z. `~OE_` deve essere attivo solo quando la scheda risponde in lettura, oppure in scrittura. **Non** va collegato a `CDONE`: il gating è ciclo per ciclo, con i segnali che il firmware già asserisce quando la scheda risponde:

  ```
  ~OE_ = NAND(PHI2, NAND(R/W_, AND3(DEV_SEL_N, EXTSEL_N, MPD)))
  ```

  Parti: un `74LVC11` e due NAND (una è la quarta porta libera di `U5` se diventa un 74LVC00, l'altra un `74LVC1G00`); `U19` pin 22 non va più a `U23` pin 4. $D1FF in lettura non abilita `U19`. I tempi vanno misurati: i tre segnali sono asseriti dall'ESP32 dopo la salita di PHI2, quindi `~OE_` si abbassa in ritardo e il dato deve comunque essere valido alla CPU prima della discesa di PHI2 meno il tempo di setup.

- **Priorità A — hold del dato sul fronte di salita di WR_n.** `~mWE` nasce da un 74HC00 a 3.3 V; lo stesso fronte di discesa di PHI2 disabilita `U19` tramite `U23`: il dato può sparire quasi nello stesso istante in cui WR_n sale. Usare una porta più veloce (74LVC00/74AHC00) e rilasciare `U19` dopo WR_n; verificare con l'analizzatore logico.

- **Priorità A — IRQ push-pull sul bus.** `~mVIRQ` arriva a `U16` e quindi al pin B del connettore ECI. La linea IRQ Atari è open-collector condivisa: serve uno stadio open-drain tollerante a 5 V (`74LVC1G07`, un componente, con `U16` pin 6 scollegato dalla rete `~IRQ`).

- **Identificazione dell'IRQ: via software, non tramite $D1FF.** Il PBI prevede che la VERA pilota D7 in lettura di $D1FF con gli altri bit Hi-Z, ma `U19` abilita tutti e otto i canali o nessuno: D0–D6 sarebbero portati sul bus a un livello casuale, in contesa con altri device PBI. Il filo `~mVIRQ`→GPIO0 e l'opzione firmware relativa sono stati eliminati. Un programma che usa gli IRQ della VERA aggancia `VIMIRQ` ($0216): legge `ISR` e `IEN`, conferma VSYNC/LINE/SPRCOL scrivendo 1 in `ISR`, maschera AFLOW in `IEN`, altrimenti passa al gestore precedente. `PDIMSK` ($0249) resta a 0. Serve solo lo stadio open-drain della voce precedente.

Sequenza di reset e configurazione

Il firmware già tiene `ARESET` e `CRESET` bassi in `setup()`, rilascia `CRESET`, attende `CDONE` e solo allora rilascia `ARESET`. La sequenza vale **solo al power-on**: non deve essere ripetuta se `CDONE` scende dopo una riconfigurazione (CTRL bit 7). `ARESET` deve essere open-drain con pull-up e il tasto RESET Atari deve continuare a funzionare. Fra `CDONE` e il rilascio di `ARESET` il firmware attende `VERA_POST_CDONE_DELAY_US` (default 1000 µs) per coprire il POR interno della VERA. Se la VERA non risponde entro ≈0.7 s, l'`INIT` della ROM PBI non scrive sul bus ed emette tre beep dall'altoparlante della console.

Modifiche al firmware ESP32

- I **registri VERA** ($D100–$D11F) rispondono con il latch a 0 (vedere sezione 2): prima il firmware asseriva `DEV_SEL_N` solo con la scheda selezionata, quindi dopo l'`INIT` i registri non erano più raggiungibili.

- Il **latch** usa solo il proprio bit (`data & 0x80`).

- Il **trace per registro** è compilato solo con `VERA_TRACE_REGS=1`: `log_send()` nel loop del bus costa ≈1 µs, più di un ciclo 6502 (559 ns).

- La **ROM 6502** (`6502/src/vera_pbi_handler.s`) è l'handler corrente: `IRQVECTOR` conferma gli IRQ, `WAIT_VERA` attende fino a ≈0.7 s il byte `'V'` del DCSEL 63 e l'`INIT` salta lo schermo se la VERA non risponde.

8 Conclusione

L'analisi conferma che l'architettura hardware e software del progetto è solida e performante. L'ESP32-S3FN8 a 240 MHz garantisce circa 67 cicli di clock nella finestra critica di PHI2 alto (279 ns), ampiamente sufficienti per l'intera catena di decodifica e pilotaggio del bus. Le tecniche di programmazione a basso livello — accesso diretto ai registri GPIO (`GPIO.in`, `GPIO.in1.val`, `GPIO.out_w1ts/w1tc`, `GPIO.out1_w1tc/out1_w1ts`), funzioni in IRAM, LUT pre-calcolata per il bus dati — garantiscono un percorso critico tipicamente inferiore a 20 cicli.

Segnali di controllo revisionati

La revisione del firmware e della mappatura hardware ha chiarito e corretto i seguenti aspetti:

- **EXTSEL_N** (GPIO 41, Dedicated GPIO output ch 1) è un segnale *per-ciclo*: viene asserito basso per gli accessi ai registri VERA $D100–$D11F (sempre), quando il device è selezionato (`selected=true`) e l'accesso ricade in $D100–$D1FE (escluso $D1FF) oppure in $D600–$D7FF, e quando RAMbo è attivo e l'accesso è nella finestra $4000–$7FFF (indipendentemente da `selected`). Viene sempre deassertato al termine del ciclo. Non è un segnale persistente legato allo stato del latch VCS.
- **MPD (Math Pack Disable)** (GPIO 42, Dedicated GPIO output ch 2) viene asserito basso esclusivamente quando il device è selezionato e l'accesso riguarda la ROM interna $D800–$DFFF. Non viene asserito per accessi alla RAM, a VERA o alla finestra RAMbo. Viene asserito insieme a EXTSEL_N e DEV_SEL_N in un'unica istruzione TIE (`cpu_ll_write_dedic_gpio_mask`).
- **RAM interna PBI 512 B** ($D600–$D7FF): indirizzata su 9 bit via `ram_pbi[addr & 0x1FFu]`. Rilevata per confronto diretto dell'indirizzo a 16 bit (`addr >= 0xD600u && addr <= 0xD7FFu`): non esiste più il segnale hardware `RAM_SEL_N`. GPIO 40 (MTDO, QFN56 pin 45) è ora `DEV_SEL_N` (VERA chip select).
- **Decodifica indirizzi software (nessuna logica 74HCT/74LVC):** il firmware precedente utilizzava IC 74HCT e 74LVC per pre-decodificare gli indirizzi e generare segnali (`D1XX_N`, `ROM_SEL_N`, `RAM_SEL_N`) come ingressi ESP32. Nel firmware attuale questi segnali sono stati eliminati: l'ESP32 legge A0–A15 direttamente (tramite i 74LVC4245 U3, U4, U6, U16 e U19, usati solo per adattamento di livello 5 V $\leftrightarrow$ 3.3 V) e decodifica tutto in software tramite `decode_addr()`.
- **Ripartizione dei transceiver (netlist 2026-09-01):** `U4` porta A0–A7 e `U3` porta A8–A15 (ingressi Atari→ESP32); `U6` porta PHI2, R/W_ (e a test point D1XX, S4, S5, CCTL, REFRESH); `U16` porta EXTSEL_N, MPD, ARESET (uscite) e l'IRQ della VERA; `U19` è il bus dati bidirezionale. Le versioni precedenti di questo documento e di `PIN-MAPPING.md` scambiavano `U6` e `U16`.
- **Snooping passivo della PIA 6520 — PBCTL ($D303) e PORTB ($D301):** il chip **6520 PIA è fisicamente presente** sulla scheda madre dell'Atari e risponde

a $D301/$D303 normalmente. Il firmware **non emula** il 6520: lo **snoopa** in modo passivo. La distinzione operativa è:

Indirizzo	Chi risponde	Ruolo ESP32
$D301 / $D303	6520 PIA (hardware reale)	Solo snooping — non pilota mai D0–D7
$4000–$7FFF	ESP32 (RAMbo window)	Attivo: EXTSEL_N + pilota D0–D7

Quando il 6502 scrive a $D303 o $D301, il 6520 riceve e processa la transazione; l'ESP32 osserva gli stessi dati sul bus e aggiorna le shadow copy locali (`PBCTL`, `PORTB`). Non vi è mai contesa sul bus perché l'ESP32 non guida il bus dati per quegli indirizzi.

La PIA espone due registri sovrapposti a $D301: il *Data Direction Register* (DDR) quando PBCTL bit 2 = 0, e l'*Output Register* (PORTB, bank selection) quando PBCTL bit 2 = 1. Il firmware snoopa $D303 ad ogni write cycle e aggiorna `PBCTL`; le write a $D301 aggiornano `PORTB` **solo** quando `PBCTL & 0x04 != 0`.

Il tracciamento di PBCTL è necessario *precisamente perché* lo snooping è passivo: dal bus, una write DDR e una write all'Output Register su $D301 sono **indistinguibili** (stesse linee, stesso ciclo). Senza lo stato di PBCTL, le write DDR della sequenza di init del sistema operativo (`0xFF` → $D301 con `PBCTL = 0`) verrebbero erroneamente interpretate come selezioni di banco. Rif.: datasheet Motorola 6520 PIA; Altirra Hardware Reference Manual §8; forum AtariAge (utente *warerat*).

- **GPIO 15 e GPIO 16** (pad RTC `XTAL_32K_P/N`): il progetto non impiega quarzo da 32 kHz; i pad sono utilizzati come normali GPIO per segnali indirizzo. Il MCU richiede invece un quarzo esterno a **40 MHz** sulle pad dedicate **XTAL_P** (pin 54) e **XTAL_N** (pin 53) — senza di esso il chip non si avvia (vedere sezione 3.2).

Il firmware supporta le due modalità (`VERA_BOARD_IS_PBI=1` PBI, `VERA_BOARD_IS_PBI=0` CCTL) con ambienti di build dedicati (`esp32s3fn8`, `esp32s3fn8-cctl`), senza duplicazione di codice. La tabella dei pin fisici del package QFN56 consente di realizzare il PCB direttamente dalla documentazione senza ambiguità tra numero GPIO logico e pad fisico.